

CustomDMVST에 ST-ResNet 도입을 위한 구현 리서치 보고서

Executive Summary

greenClothesZelda/CustomDMVST의 `Cosine_base_IR` 브랜치는 현재 (1) **causal cosine-similarity** 기반 retrieval(`IRModule`) 과 (2) **spatial attention + temporal BiLSTM** 기반 **neural forecasting**(`IRVSTNet`) 을 gate로 섞어 `(B, N)` 다음 시점 수요를 예측합니다. ① 이 구조를 **ST-ResNet(arXiv:1610.00081)** 의 “closeness/period/trend + residual CNN + parametric fusion + external factors” 구조로 교체하려면, **모델 파일 교체만으로는 부족하고 데이터셋(시계열 keyframe 구성)까지 함께 변경되어야 합니다.** ② ③

핵심 구현 전략은 다음과 같습니다.

- **DMVSTLoss는 그대로 유지**하되, ST-ResNet의 원 논문 기본 세팅(출력 `tanh` 로 `[-1,1]`)은 현재 코드의 “수요=비음수 카운트” 및 loss 형태와 충돌 가능성이 커서, **출력 활성/스케일링은 repo의 loss/평가지표에 맞게 조정**합니다. ④ ⑤
- **Trainer(Transformers) 연동은 유지**하되, 모델 출력 dict에서 `loss` 를 제외한 값들이 evaluation logits로 수집되는 Trainer 동작을 활용하여(키 이름이 `predictions` 여도 동작), 인터페이스를 크게 깨지 않도록 합니다. ⑥ ⑦
- 데이터는 현재 `(N, T)` window를 쓰지만, ST-ResNet은 **closeness/period/trend keyframe**(최근/하루 주기/주간 추세)을 별도로 구성합니다. 따라서 `MyDataset` 을 **keyframe 기반으로 재정의**하고, 모델 입력 `demands_series` 의 의미를 “(L_total, H, W) 또는 (L_total, ...) 프레임 스택”으로 바꾸되, **최종 예측은 기존과 동일하게** `(B, N)` 로 내보내 DMVSTLoss·테스트 루프의 인터페이스를 보존하는 방식이 가장 안전합니다. ② ⑧ ⑤

아래 보고서에는 (1) 아키텍처-코드 매핑, (2) 코드 수준 패치/의사코드, (3) 데이터 로더 전처리 변경, (4) 학습/추론 통합, (5) 테스트 계획, (6) 함정/디버깅 체크리스트, (7) 일정/마일스톤, (8) 비교 표(파라미터·FLOPs·메모리), (9) 핵심 코드 스니펫과 Mermaid 다이어그램을 포함합니다.

오픈 아이템(확정 필요): `grid(shape)`, 시계열 간격(시간/15분 등), 수요 채널 수(1 vs inflow/outflow 2), `grid(7000).npy` 의 실제 공간 해상도(H,W), 기상 CSV와 `grid` 시계열 정렬 방식, holiday/event feature 유무, 학습 하이퍼파라미터(ResUnit 수 등). ② ⑨

코드베이스 현황 분석

진입점과 학습 파이프라인

현재 학습은 `main.py` 에서 Hydra 설정을 읽어 데이터셋을 만들고, `transformers.Trainer` 로 학습합니다. 핵심 흐름은 다음과 같습니다. ④

- 데이터셋: `models/loader/DemandDataset.py` 의 `MyDataset`
- 모델: `models/GNVSTNet.py` 의 `IRModule` (retrieval) + `IRVSTNet` (attention+BiLSTM)
- 래퍼: `ModelTrainer` 가 HuggingFace Trainer 입력 포맷(`labels` 포함)과 loss 계산을 맞춥니다. ⑥

- 손실: `main.py` 내부 `DMVSTLoss` (제공 오차 기반 + 상대항 가중) 그대로 사용 중이며, `trainer_model = ModelTrainer(model, loss=DMVSTLoss(...))` 형태로 주입됩니다. 4

또한 `docs/pipeline.md` 가 이 브랜치의 데이터 흐름, split, retrieval 동작, 모델 fusion 구조를 비교적 정확히 문서화하고 있습니다. 1

데이터셋의 현재 출력 규격

`MyDataset.__getitem__` 은 다음 dict를 반환하고, `collate_fn` 이 key별 stack으로 배치를 만듭니다. 2

- `demands_series`: (N, T) (현재는 `demand_arr[index:index+time_step]` 를 전치)
- `labels`: (N,) (다음 시점)
- `time`: (8,) (요일 one-hot 7 + hour/24 1)
- `weather`: (F,) (기본 4개 컬럼)
- `sample_idx`: scalar (index)

`grid(size).npy` 를 로드한 뒤 (T, -1) 로 flatten하고, 총 수요가 큰 상위 `num_nodes` 만 남겨 `demand_arr` 를 만듭니다. 즉 공간 구조(H×W)는 내부적으로 폐기되고 “선택된 N개 cell의 벡터 시계열”로 모델링 됩니다. 2

기존 모델(IRVSTNet)과 IRModule 의존성

`IRVSTNet` 은 demand/node/time/weather를 공통 차원으로 임베딩한 뒤, time-slice별 node self-attention → node별 BiLSTM → linear로 neural prediction을 만들고, `IRModule` 의 retrieval prediction과 `gate(lambda_layer)` 로 섞어 (B,N) 을 출력합니다. 6 1

따라서 **ST-ResNet으로 교체 시 IRModule·gate 구조를 유지할지/제거할지**가 큰 설계 포인트인데, 사용자 요청은 “ST-ResNet으로 교체 + DMVSTLoss 유지”이므로, 본 보고서는 **IRModule 제거(또는 비활성화) 방향**을 기본안으로 제안합니다. IRModule은 데이터셋의 `demands_series` 가 (N,T) 라는 가정에 강결합되어 있어, ST-ResNet 입력을 위해 `demands_series` 의미를 바꾸면 IRModule도 함께 깨질 가능성이 큼니다. 6 2

ST-ResNet 핵심 구조 정리

ST-ResNet은 arXiv 10 논문(arXiv:1610.00081)에서 제안된 “도시 격자(I×J)에서 inflow/outflow(2채널)의 다음 시점 텐서 예측” 모델입니다. 저자는 Junbo Zhang 11, Yu Zheng 12, Dekang Qi 13 입니다. 3

입력 텐서와 4개 컴포넌트

논문은 시계열을 다음 4개 컴포넌트로 나눕니다. 14

- **Temporal closeness**: 최근 몇 개 구간의 flow map들
- **Period**: 하루 주기(예: 24시간)로 반복되는 패턴(“전날 같은 시간”)
- **Trend**: 주간/장기 추세(“지난주 같은 시간”)
- **External factors**: 날씨/요일/휴일/이벤트 등 외부 요인(2-layer FC로 map으로 투영)

closeness/period/trend 3개 컴포넌트는 동일한 backbone을 공유(구조 동일, 파라미터는 별도)하며, 각 컴포넌트는 “Conv → Residual unit stack → Conv”로 구성됩니다. 5

Residual unit과 fusion 방식

Residual unit은 일반적인 residual learning 형태를 따릅니다. ¹⁵

- 입력 $X^{(l)}$ 에 대해 $X^{(l+1)} = X^{(l)} + F(X^{(l)})$

3개 branch의 최종 feature map 출력은 **지역(region)·채널별로 다른 가중치 행렬(learnable parametric matrices)** 로 element-wise 가중합되어 합쳐집니다. ¹⁶

- $X_Res = W_c \circ X_c + W_p \circ X_p + W_q \circ X_q$ ($\circ$: Hadamard/element-wise)

External component 출력 X_Ext 와 합친 뒤, 논문에서는 $\tanh(X_Res + X_Ext)$ 로 [-1,1] 범위로 매핑하여 수렴을 돕는다고 설명합니다. ¹⁶

원 논문 하이퍼파라미터(대표값)

논문(및 확장 저널 버전)에서 제시된 대표 설정은 다음과 같은 형태로 요약됩니다. ¹⁶

- Conv/ResUnit: 3×3 conv, 필터 64
- 최종 Conv2: 출력 채널(원 논문은 inflow/outflow=2)
- period 간격 p 는 1일, trend 간격 q 는 1주를 고정하는 식의 선택
- external feature는 one-hot 및 min-max 정규화를 활용

본 repo에 적용할 때는 (a) 현재 데이터가 **단일 수요 채널일** 가능성, (b) 손실이 DMVSTLoss로 고정, (c) trainer가 non-negative 예측을 기대(실제로 `ModelTrainer` 가 `ReLU` 를 적용)라는 점 때문에, **출력 활성화·정규화**는 “논문 그대로”가 아니라 “**현 파이프라인과 일관되게**” 조정하는 것이 실무적으로 안전합니다. ^{6 4 16}

(참고) 한국어 요약 자료로는 ST-ResNet 구조를 closeness/period/trend로 분리하고 parametric fusion을 강조하는 리뷰가 존재합니다(비공식 자료). ¹⁷

ST-ResNet 컴포넌트와 CustomDMVST 코드 매핑

변경 대상 파일과 역할 매핑

아래는 “ST-ResNet 구성요소 → CustomDMVST에서 대응되는 구현 위치”를 최소 변경 관점에서 매핑한 것입니다(핵심만).

ST-ResNet 컴포넌트	CustomDMVST에서의 구현/수정 위치	수정 종류
Closeness branch (Conv + ResUnits + Conv)	<code>models/</code> 아래 신규 파일 예: <code>models/STResNet.py</code>	신규 추가
Period branch	동일(<code>models/STResNet.py</code>)	신규 추가
Trend branch	동일(<code>models/STResNet.py</code>)	신규 추가
Parametric-matrix fusion (W_c, W_p, W_t)	동일(<code>models/STResNet.py</code>)	신규 추가
External component (2-layer FC)	동일 + dataset에서 외부 feature 제공 (<code>weather</code> , <code>time</code>)	신규 추가 + Dataset 정규화/차원 정의

ST-ResNet 컴포넌트	CustomDMVST에서의 구현/수정 위치	수정 종류
Keyframe 생성(최근/주기/추세 시점 샘플링)	<code>models/loader/DemandDataset.py</code> 의 <code>MyDataset</code>	대수정
학습 인덱스 split (warmup 제거/대체)	<code>main.py</code>	수정
Trainer wrapper + DMVSTLoss 유지	<code>models/GNVSTNet.py</code> 의 <code>ModelTrainer</code> , <code>main.py</code> 의 <code>DMVSTLoss</code>	유지(단, 입력 규격 반영 위해 model forward만 교체)
추론/평가(test_loop)에서 full grid reconstruction	<code>runners/test.py</code>	dataset의 <code>get_full_labels</code> 로직 변경에 맞춰 수정 가능성

현 상태에서 학습 파이프라인은 `main.py` 가 `IRModule` 과 `IRVSTNet` 을 명시적으로 생성합니다. ST-ResNet 교체 시 이 부분을 ST-ResNet 인스턴스로 바꾸고, `IRModule` 관련 warmup 로직을 제거해야 합니다. 4 9

Mermaid: 컴포넌트 매핑 다이어그램

```

flowchart LR
    subgraph Repo[greenClothesZelda/CustomDMVST | Cosine_base_IR]
        MAIN[main.py] --> DS[MyDataset<br/>models/loader/DemandDataset.py]
        MAIN --> WRAP[ModelTrainer<br/>models/GNVSTNet.py]
        MAIN --> CFG[configs/cosine_base_ir.yaml]
        WRAP --> LOSS[DMVSTLoss<br/>(keep formula/interface)]
    end

    subgraph Replace[ST-ResNet Replacement]
        DS2[MyDataset (modified)<br/>keyframe: closeness/period/trend] --> STIN[demands_series: (B,L,H,W)]
        STIN --> ST[STResNet<br/>models/STResNet.py]
        DS2 --> EXT[weather/time external features]
        EXT --> ST
        ST --> OUT[(pred: (B,N))]
        OUT --> LOSS
        LOSS --> WRAP
    end

    MAIN -.replace IRModule+IRVSTNet.-> ST
    DS -.modify.-> DS2
    CFG -.add model.STResNet config.-> ST

```

구현 계획과 코드 수준 설계

설계 원칙과 목표 I/O 규격

목표는 “학습 루프/손실/평가 최대한 유지 + 모델만 ST-ResNet으로 교체”입니다.

- **Trainer 입력 dict 키는 그대로 유지:**

`demands_series, weather, time, sample_idx, labels` (변경 최소화) ④

- 단, `demands_series` 의 **shape/의미를 변경:**

- 기존: `(B, N, T)` (N개 노드, T 길이) ⑥

- 제안: `(B, L_total, H, W)` (ST-ResNet에서 keyframes를 channel 축으로 concat한 텐서) ⑤

- 모델 출력: **기존과 동일하게** `(B, N)`

- ST-ResNet이 내부적으로 `(B, 1(or2), H, W)` 를 만들더라도 마지막에 `retained_flat_indices` 로 gather하여 `(B, N)` 만 내보냅니다. ⑧

- dtype:

- 입력 수요는 정수(long)로 저장돼도 모델 내부에서는 `float32` 로 캐스팅해 conv 연산

- `weather/time` 은 `float32`

- `labels` 는 `float32` 로 loss 계산(현 코드는 `labels.to(torch.float32)` 적용) ⑥

- device:

- Trainer가 배치를 device로 올리므로, model 내부에서 buffer/index 텐서를 같은 device로 맞춰야 합니다.

단계별 구현 플랜

단계 A: ST-ResNet 모델 파일 추가

신규 파일 예시: `models/STResNet.py`

핵심 클래스 구성(권장):

- `ResidualUnit2D(nb_filter)`
- `STResBranch(in_channels, nb_filter, nb_flow, nb_residual_unit)`
- `ExternalFC(external_dim, hidden_dim, out_dim=nb_flow*H*W)`
- `STResNet(...)` (세 branch + fusion weight + external + gather)

핵심 포인트는 “Trainer wrapper(`ModelTrainer`)는 유지하고, 내부 model만 교체”이므로, `STResNet`의 `forward(demands_series, weather, time, sample_idx)` 시그니처를 맞추되, `sample_idx` 는 사용하지 않아도 됩니다. ⑥

코드 스니펫: 모델 클래스 골격 (핵심만)

```
# models/STResNet.py (new)

import torch
import torch.nn as nn
import torch.nn.functional as F

class ResidualUnit2D(nn.Module):
    def __init__(self, nb_filter: int):
        super().__init__()
        self.conv1 = nn.Conv2d(nb_filter, nb_filter, kernel_size=3, padding=1)
```

```

self.conv2 = nn.Conv2d(nb_filter, nb_filter, kernel_size=3, padding=1)

def forward(self, x):
    h = F.relu(x)
    h = self.conv1(h)
    h = F.relu(h)
    h = self.conv2(h)
    return x + h # identity skip

class STResBranch(nn.Module):
    def __init__(self, in_channels, nb_filter, nb_flow, nb_residual_unit):
        super().__init__()
        self.conv_in = nn.Conv2d(in_channels, nb_filter, kernel_size=3, padding=1)
        self.res_units = nn.Sequential(*[ResidualUnit2D(nb_filter) for _ in
range(nb_residual_unit)])
        self.conv_out = nn.Conv2d(nb_filter, nb_flow, kernel_size=3, padding=1)

    def forward(self, x):
        # x: (B, in_channels, H, W)
        h = self.conv_in(x)
        h = self.res_units(h)
        y = self.conv_out(h) # (B, nb_flow, H, W)
        return y

class STResNet(nn.Module):
    def __init__(self, H, W, retained_flat_indices,
        nb_flow=1,
        len_c=3, len_p=1, len_t=1,
        nb_filter=64, nb_residual_unit=12,
        external_dim=0, external_hidden=64,
        use_external=True):
        super().__init__()
        self.H, self.W = H, W
        self.nb_flow = nb_flow
        self.len_c, self.len_p, self.len_t = len_c, len_p, len_t
        self.use_external = use_external and (external_dim > 0)

        # register indices as buffer to be checkpointed and moved with .to(device)
        self.register_buffer("retained_flat_indices", retained_flat_indices.to(torch.long))

        # branches (skip creation if length=0)
        if len_c > 0:
            self.branch_c = STResBranch(in_channels=len_c * nb_flow, nb_filter=nb_filter,
                nb_flow=nb_flow, nb_residual_unit=nb_residual_unit)
        else:
            self.branch_c = None
        if len_p > 0:
            self.branch_p = STResBranch(in_channels=len_p * nb_flow, nb_filter=nb_filter,
                nb_flow=nb_flow, nb_residual_unit=nb_residual_unit)
        else:
            self.branch_p = None

```

```

if len_t > 0:
    self.branch_t = STResBranch(in_channels=len_t * nb_flow, nb_filter=nb_filter,
                                nb_flow=nb_flow, nb_residual_unit=nb_residual_unit)
else:
    self.branch_t = None

# parametric fusion weights: (nb_flow, H, W)
if len_c > 0:
    self.Wc = nn.Parameter(torch.ones(nb_flow, H, W))
if len_p > 0:
    self.Wp = nn.Parameter(torch.ones(nb_flow, H, W))
if len_t > 0:
    self.Wt = nn.Parameter(torch.ones(nb_flow, H, W))

if self.use_external:
    self.ext_fc1 = nn.Linear(external_dim, external_hidden)
    self.ext_fc2 = nn.Linear(external_hidden, nb_flow * H * W)

def forward(self, demands_series, weather, time, sample_idx=None):
    """
    demands_series: (B, L_total, H, W) where L_total = len_c + len_p + len_t (for nb_flow=1)
    weather: (B, F), time: (B, 8)
    returns: (B, N_selected)
    """
    B = demands_series.size(0)
    x = demands_series.to(torch.float32)

    # build (B, C_in, H, W) by treating L_total as channel when nb_flow=1
    # If nb_flow>1, dataset should already pack frames accordingly.
    # Here: nb_flow=1 assumption -> x is (B, L_total, H, W)
    pos = 0
    out = 0

    if self.branch_c is not None:
        xc = x[:, pos:pos+self.len_c, :, :]
        pos += self.len_c
        yc = self.branch_c(xc)
        out = out + self.Wc.unsqueeze(0) * yc

    if self.branch_p is not None:
        xp = x[:, pos:pos+self.len_p, :, :]
        pos += self.len_p
        yp = self.branch_p(xp)
        out = out + self.Wp.unsqueeze(0) * yp

    if self.branch_t is not None:
        xt = x[:, pos:pos+self.len_t, :, :]
        pos += self.len_t
        yt = self.branch_t(xt)
        out = out + self.Wt.unsqueeze(0) * yt

```

```

if self.use_external:
    ext = torch.cat([weather.to(torch.float32), time.to(torch.float32)], dim=1)
    h = F.relu(self.ext_fc1(ext))
    x_ext = self.ext_fc2(h).view(B, self.nb_flow, self.H, self.W)
    out = out + x_ext

# IMPORTANT: 기존 repo는 비음수 예측을 선호(Wrapper에서 ReLU)
# out = torch.tanh(out) # paper-style, but may conflict with DMVSTLoss scale
# leave out raw; ModelTrainer will apply ReLU for returned predictions.
out_flat = out.view(B, self.nb_flow, -1)[: , 0, :] # (B, H*W)

# gather selected nodes -> (B, N)
pred_nodes = out_flat.index_select(dim=1, index=self.retained_flat_indices)
return pred_nodes

```

이 설계는 논문의 “3개 branch + parametric fusion + external” 핵심을 구현하면서도, 현재 repo의 (B, N) 출력과 loss/평가 흐름을 깨지 않도록 합니다. 5 8

단계 B: Dataset을 ST-ResNet keyframe 생성 방식으로 변경

현재 dataset은 단순 window (index:index+time_step) 만 만들지만, ST-ResNet은 다음 시점 t에 대해 keyframe들을 선택합니다. 5

- closeness: t-1, t-2, ..., t-1_c
- period: t-p, t-2p, ... (보통 p=하루 간격)
- trend: t-q, t-2q, ... (보통 q=1주 간격)

오픈 아이템: 이 repo 데이터의 time resolution이 1시간으로 가정될 근거(시간 feature 생성 방식, 24시간 가정)는 있지만, 확정은 필요합니다. 2

권장 변경: MyDataset 이 반환하는 demands_series 를 (N,T) → (L_total,H,W)로

현재 grid(size).numpy 는 로드 직후 reshape로 flatten됩니다. ST-ResNet conv를 쓰려면, 최소한 원래의 2D grid 형태(H,W)를 유지해야 합니다. 따라서 np.load(...) 직후 텐서 shape를 확인해서 다음을 저장하세요.

- self.grid_raw : (T, H, W) 또는 (T, C, H, W) (오픈 아이템: 실제 파일 shape)
- self.origin_demand_arr : 기존처럼 (T, H*W) flatten도 병행 저장(평가 및 top-k 선택, region_mean 계산에 유용) 8

그리고 __getitem__(i) 에서 target time index를 t = i + base_offset 으로 두고, base_offset = max(len_c, len_p*P, len_t*Q) 처럼 “필요한 최대 과거 참조”를 만족하도록 구성하는 것이 안전합니다(기존 warmup과 개념적으로 유사하지만 retrieval이 아니라 keyframe 구성 때문). 1 5

의사코드: Dataset keyframe 생성

```

# models/loader/DemandDataset.py (modify)

class MyDataset(Dataset):
    def __init__(..., st_resnet):
        # st_resnet.len_c, len_p, len_t, period_interval, trend_interval 등 config에서 받기

```



```

self.len_c = st_resnet.len_c
self.len_p = st_resnet.len_p
self.len_t = st_resnet.len_t
self.period_interval = st_resnet.period_interval # e.g., 24
self.trend_interval = st_resnet.trend_interval # e.g., 168

self.base_offset = max(
    self.len_c,
    self.len_p * self.period_interval,
    self.len_t * self.trend_interval
)

grid = np.load(root/f"grid({size}).npy")
# CASE1: grid shape (T,H,W)
# CASE2: grid shape (T,C,H,W)
# -> normalize to (T, nb_flow, H, W)
self.grid = torch.from_numpy(grid) # dtype check
...

def __len__(self):
    # target time t must exist
    return T - self.base_offset

def __getitem__(self, i):
    t = i + self.base_offset # target

    frames = []
    # closeness
    for k in range(1, self.len_c+1):
        frames.append(self.grid[t-k]) # (nb_flow,H,W)
    # period
    for k in range(1, self.len_p+1):
        frames.append(self.grid[t-k*self.period_interval])
    # trend
    for k in range(1, self.len_t+1):
        frames.append(self.grid[t-k*self.trend_interval])

    demands_series = torch.cat(frames, dim=0) # (L_total*nb_flow, H, W) OR if nb_flow=1 ->
    (L_total, H, W)
    labels_full = self.origin_demand_arr[t] # (H*W,)
    labels = labels_full[self.retained_flat_indices] # (N,)
    return {
        "demands_series": demands_series,
        "labels": labels,
        "weather": weather[t],
        "time": time[t],
        "sample_idx": torch.tensor(i) # dataset-local index
    }

```

단계 C: main.py에서 모델 초기화/분할 로직 교체

main.py 는 currently IModule 을 만들어 warmup을 k 로 두고 train indices를 [k, train_end) 로 잡습니다. ST-ResNet 교체 시에는 k 가 없어지므로, 다음 중 하나가 필요합니다. ⁴

- 옵션 1(권장): dataset이 base_offset 을 반영하여 __len__ 이 이미 줄어들게 만들고, train/test split은 단순히 [0, train_end) / [train_end, len(dataset)) 로 처리
- 옵션 2: dataset 길이는 그대로 두되, train_start = base_offset 으로 warmup처럼 index 범위 제한 (실수 가능성 ↑)

패치 방향(옵션 1) 예시

```
--- a/main.py
+++ b/main.py
@@
-from models.GNVSTNet import IModule, IRVSTNet, ModelTrainer, collate_fn
+from models.GNVSTNet import ModelTrainer, collate_fn
+from models.STResNet import STResNet

@@ def run(config):
- warmup = config.model.IModule.k
- if train_end <= warmup:
-     raise ValueError(...)
- train_indices = list(range(warmup, train_end))
+ # ST-ResNet: dataset 자체가 base_offset을 반영해 length를 정의(권장)
+ train_indices = list(range(0, train_end))

@@
- ir_module = IModule(dataset, device, k=config.model.IModule.k)
- model = IRVSTNet(ir_module=ir_module, **config.model['IRVSTNet'])
+ # dataset이 공간 크기/retained indices 제공하도록 확장
+ model = STResNet(
+     H=dataset.grid_H,
+     W=dataset.grid_W,
+     retained_flat_indices=dataset.retained_flat_indices,
+     **config.model['STResNet'],
+     external_dim=dataset.weather_list.shape[1] + 8
+ )

    trainer_model = ModelTrainer(model,
    loss=DMVSTLoss(lambda_rel=1)).to(device)
```

이때 config도 바뀌어야 하며, configs/cosine_base_ir.yaml 에 model.STResNet 블록을 추가하고, model.IModule, model.IRVSTNet 은 제거하거나 비활성화합니다. ⁹

단계 D: Training/Checkpoint/Optimizer hooks 호환성

이 repo는 Hugging Face ¹⁸ Trainer 를 사용합니다. Trainer는 model output이 dict일 때 loss 키를 제외한 값들을 logits로 수집할 수 있습니다(즉 key 이름이 꼭 logits 일 필요는 없음). ⁷

현재 `ModelTrainer.forward` 는 `{'predictions': predictions, 'loss': loss}` 형태로 dict를 리턴합니다. (또한 반환 직전에 `ReLU` 로 predictions를 비음수화) ⁶
따라서 ST-ResNet 도입 후에도 **ModelTrainer**는 그대로 사용해도 됩니다(내부 `self.model(...)` 만 STResNet으로 교체).

체크포인트:

- Trainer는 `state_dict` 기반으로 저장하므로, STResNet 내부에서 `retained_flat_indices` 같은 인덱스 텐서를 `register_buffer` 로 등록하면 자동 포함됩니다(학습 재개 시 shape 불일치 디버깅에 매우 유리). ¹⁹

AMP/FP16:

- TrainingArguments로 fp16/bf16 옵션을 켤 수 있으나, DMVSTLoss가 `diff**2` 를 수행하므로 fp16에서 overflow 가능성이 있습니다. 안전하게 하려면 (1) fp16을 끄거나, (2) loss 내부 산술을 float32로 올려 계산하는 보강이 필요합니다(공식 Trainer 문서에서도 커스텀 loss 시 dtype/labels 처리에 유의하도록 안내). ⁴ ²⁰

멀티 GPU:

- Trainer는 DDP/Accelerate를 사용하므로, 모델 forward가 “입력 텐서 device를 신뢰”하고 `.to(self.device)` 같은 강제 이동을 최소화하는 편이 안전합니다. 본 repo 코드도 배치에서 `.to(device)` 를 수행합니다. ⁸

데이터 전처리/로더 변경 목록

요구사항(사용자 요청 #3)에 맞춰, 변경 항목을 “필수” 위주로 정리합니다.

- **Sequence length 재정의**
- 기존 `dataset.time_step = 24` 는 window 길이였으나, ST-ResNet은 `len_c, len_p, len_t` 와 `period_interval, trend_interval` 이 핵심입니다. (예: hour-based면 `period_interval=24`, `trend_interval=168`) ¹⁶
- 따라서 config에 `dataset.time_step` 만 두면 의미가 바뀌므로, `model.STResNet` 에 명시적으로 추가(권장).
- **Keyframe 샘플링 로직**
- `__getitem__` 에서 target 시간 t를 기준으로 필요한 과거 시점 프레임을 수집.
- boundary 처리를 위해 `base_offset` 을 도입하고 `__len__` 을 줄이는 형태가 가장 견고.
- **공간 텐서 복원**
- 현재는 grid를 flatten하고 top-k cell을 선택해 공간 구조를 잃습니다. ST-ResNet은 Conv2D 기반이므로 최소한 `(H,W)` 구조를 살려야 합니다. ² ⁵
- top-k 평가를 유지하려면 “전체 grid 기반 conv → 마지막에 top-k만 gather” 방식으로 잇는 것이 변경 범위를 최소화합니다.
- **Normalization**
- 논문은 external feature(온도/풍속 등)에 min-max, categorical에는 one-hot 등을 사용합니다. ¹⁶
- 이 repo는 현재 weather를 원 값 그대로 Linear로 임베딩합니다. ST-ResNet external FC는 스케일 민감도가 더 크므로:
 - weather: `StandardScaler` (mean/std) 또는 min-max(0~1) 권장
 - time: 현재 7 one-hot + hour/24는 그대로 사용 가능
- 단, labels/예측 스케일을 정규화하면 DMVSTLoss의 분모(`1+y_true`)와 충돌할 수 있어, labels 스케일은 유지하고 input만 정규화하거나(권장) loss를 수정해야 하는데, 후자는 사용자 요구에 반합니다. ⁴
- **날씨/시간 길이 정렬**

- 현재 `time` 생성은 `weather.shape[0]//(7*24)` 로 repeat 후 view하는 방식이라, 길이가 7*24로 나누어떨어지지 않으면 truncation/불일치 가능성이 있습니다(잠재 버그). ST-ResNet은 period/trend를 쓰므로 이 정렬이 더 중요해집니다. 2

학습/추론 통합 단계

요구사항(사용자 요청 #4)에 맞춰 “학습 루프 변화”를 구체적으로 나열합니다.

- 학습 시작 전
- `configs/cosine_base_ir.yaml` 에 `model.STResNet` 블록 추가(아래 예시 참고)
- `DemandDataset.MyDataset` 이 `STResNet` 설정을 받아 `base_offset`을 계산하도록 변경
- 학습 중(Trainer)
- `ModelTrainer.forward` 는 그대로 두고, `self.model(demands_series, weather, time, sample_idx)` 가 STResNet을 호출하도록 `main.py`에서 생성한 model만 바꿉니다. 6
- `labels` 는 그대로 `(B,N)` 로 들어오고, DMVSTLoss로 backprop.
- 평가/추론
- `runners/test.py` 는 model 호출 시그니처가 동일하면 대부분 유지됩니다. 다만 `get_full_labels` 가 dataset index→target time 매핑을 바꿔야 하므로 해당 부분은 dataset 수정에 맞춰 업데이트 필요합니다. 8
- 멀티 GPU/AMP
- 공통적으로 “인덱스 buffer 이동, float32 loss”를 체크합니다. 21

config 예시

`configs/stresnet_base.yaml` 같은 새 config를 추가하거나, `cosine_base_ir.yaml` 을 수정합니다. 최소 config 예시는 아래와 같습니다(오픈 아이템 기반으로 조정 필요). 9

```
model:
  STResNet:
    nb_flow: 1
    len_c: 3
    len_p: 1
    len_t: 1
    period_interval: 24
    trend_interval: 168
    nb_filter: 64
    nb_residual_unit: 12
    use_external: true
    external_hidden: 64
```

테스트 설계

요구사항(사용자 요청 #5)에 맞춰 **unit + integration**으로 나눕니다. 본 repo에는 기본적으로 테스트 폴더가 없으므로 `tests/` 를 신설하는 전제를 둡니다.

Unit tests

Dataset keyframe 정확성

케이스: synthetic grid에서 “시간 t의 값이 t 자체”가 되도록 만든 뒤, 샘플 `i` 가 target `t=i+base_offset` 을 올바르게 참조하는지 확인합니다.

- 준비: `grid[t,:,:] = t` (모든 셀 동일), `meteorological_data.csv` 도 길이 T로 dummy 생성
- 설정: `len_c=3, len_p=1, len_t=1, period_interval=4, trend_interval=8` 처럼 작은 값으로 테스트
- 기대:
 - `demands_series` shape = `(len_c+len_p+len_t, H, W)`
 - `demands_series[0]` 는 `t-1`, `demands_series[1]` 는 `t-2` ... (정의한 순서대로)
 - `labels` 는 retained index에 대한 `t` 값(즉 모두 t)

예상 shape/값 예시:

- `H=2, W=3, N=2`(top-k indices 0,1 가정)
- `i=0, base_offset=8 → t=8`
- `demands_series[0,:,:] == 7`
- period frame `== 4`
- trend frame `== 0`
- `labels == [8,8]`

모델 forward shape/gradient

- 입력: `demands_series` 임의 텐서 `(B,L,H,W)`, `weather/time` 임의 텐서
- 기대:
 - output shape `(B,N)`
 - dtype float32 (또는 amp 컷을 때 mixed)
 - `output.sum().backward()` 가 에러 없이 수행

ModelTrainer 호환성

- `ModelTrainer(model=STResNet, loss=DMVSTLoss(...))` 호출 후:
- `outputs = trainer_model(...)`
- `outputs['loss']` 는 scalar tensor
- `outputs['predictions']` shape `(B,N)` 이며 ReLU 적용으로 음수 없음 ⁶

Integration tests

1-step smoke training

readme에 있는 CPU 스모크 테스트 패턴을 ST-ResNet config로 빠르게 수행합니다. 기존 스모크 패턴 자체는 Hydra/Trainer 파이프라인이 정상인지 확인하는 데 유효합니다. ²²

예: `max_steps=1, eval_steps=1, save_steps=1`

test_loop 실행

학습 직후 최소 checkpoint에서 `test_loop` 가 끝까지 돌고, 결과 csv/plot을 생성하는지 확인합니다. ⁸

테스트 실행 명령

브랜치 전환

```
git clone https://github.com/greenClothesZelda/CustomDMVST.git
cd CustomDMVST
git checkout Cosine_base_IR
```

(권장) 테스트 의존성 설치 후

```
pytest -q
```

CPU 스모크 테스트(기존 readme 패턴 참고)

```
python main.py --config-name stresnet_base \
device=cpu \
+train.use_cpu=true \
+train.max_steps=1 \
train.eval_strategy=steps \
+train.eval_steps=1 \
train.save_strategy=steps \
+train.save_steps=1 \
train.logging_steps=1
```

잠재적 함정과 디버깅 체크리스트

가장 흔한 실패 지점

- **grid 파일 shape 오해**
 - (T,H,W) 인지 (T,C,H,W) 인지, 혹은 다른 축 구조인지 확정 필요.
 - 잘못 해석하면 conv in_channels가 맞지 않아 즉시 크래시.
- **time/weather 길이 불일치**
 - 현재 time 생성 로직이 “7*24로 나누어 떨어지는 길이”를 암묵적으로 가정합니다. 길이 mismatch는 external feature alignment를 깨뜨립니다. ²
- **period/trend 인덱싱 오류**
 - base_offset / __len__ / sample_idx / get_full_labels 의 index mapping이 서로 다른 기준을 쓰면, 학습은 돌아가도 평가가 틀어집니다. ⁸
- **대용량 H×W로 인한 OOM**
 - ST-ResNet은 feature map이 (B,64,H,W) 로 여러 층 유지되므로 H,W가 크면 매우 빨리 메모리 한계를 넘습니다. (현재 config의 size=7000 가 H/W와 직접 관계가 있는지 확정 필요) ⁹
- **AMP에서 DMVSTLoss overflow**
 - diff**2 가 fp16에서 overflow할 수 있습니다(특히 demand scale이 큰 경우). ⁴

디버깅 체크리스트

- (데이터) grid.shape, T, weather.shape[0], time.shape[0] 를 startup log로 강제 출력
- (인덱스) 샘플 i에 대해 target t, closeness/period/trend 참조 t-k가 기대대로인지 단일 샘플 프린트/assert
- (모델) branch 입력 채널 수 len_* * nb_flow 가 실제 demands_series 채널과 일치하는지 assert
- (학습) 첫 step에서 loss가 NaN인지(정규화/overflow) 확인
- (평가) test_loop 에서 all_predictions.shape == all_labels.shape == (num_samples, N) 확인 ⁸
- (성능) GPU 메모리 사용량을 batch size, nb_filter, nb_residual_unit, H,W에 대해 스윙

노력 추정과 마일스톤

아래는 “코드 작성/디버깅 포함” 기준의 실무적 일정 예시(1인 기준)입니다. 데이터 형태가 복잡하거나 grid 크기가 커서 모델 축소/타일링이 필요하면 추가 시간이 듭니다(오픈 아이템).

마일스톤	산출물	예상 소요
Repo 이해/현 파이프라인 재현	기존 <code>Cosine_base_IR</code> smoke test 통과	0.5~1일 ²²
Dataset keyframe 구현	수정된 <code>MyDataset</code> + unit test 통과	1~2일
STResNet 모델 구현	<code>models/STResNet.py</code> + forward/grad unit test	1~2일
main.py/config 통합	<code>--config-name stresnet_base</code> 로 1-step 학습/평가	0.5~1일
OOM/성능 튜닝	batch/nb_filter/res_units 조정 가이드	1~3일
실험/문서화	결과 리포트(지표, 그래프), doc 업데이트	1~2일

총합: 대략 5~10 작업일(데이터 shape·해상도에 따라 변동).

원 모델 vs ST-ResNet 비교

아래 비교는 “현 repo 기본 config(embedding_dim=32, LSTM hidden=32, num_layers=3 등)”와 “ST-ResNet 대표 설정(nb_filter=64, res_units=12)”을 기준으로 한 구조적 비교입니다. 수치(Params/FLOPs/메모리)는 **H,W,max_demand,nb_flow**가 오픈 아이템이라, (a) 공식적 수식/근사 및 (b) 32×32 예시 값을 함께 제공합니다. ⁹

¹⁶

구조 비교

항목	기존(IRVSTNet + IRModule)	ST-ResNet(교체안)
핵심 공간 모델링	node self-attention(벡터)	2D CNN (격자) ¹⁵
핵심 시간 모델링	BiLSTM (긴 window)	closeness/period/trend keyframe 선택 ⁵
External	weather/time embedding 후 합산	external FC로 map 투영 후 더함 ⁵
Retrieval	cosine top-k retrieval	(기본안) 제거
출력	<code>(B,N)</code>	내부 <code>(B,1,H,W)</code> → <code>(B,N)</code> gather
학습	Trainer + DMVSTLoss	동일(손실 유지) ⁴

파라미터 수 근사

ST-ResNet(한 branch) 파라미터 근사

- Conv1: `nb_filter * (in_ch * 3*3 + 1)`
- ResUnit(2 conv) 1개: `2 * nb_filter * (nb_filter * 3*3 + 1)`
- Conv2: `nb_flow * (nb_filter * 3*3 + 1)`
- 총(3 branch): 위 ×3

- Fusion weight: $3 * nb_flow * H * W$ (Wc,Wp,Wt) ¹⁶
- External FC: $external_dim \rightarrow hidden \rightarrow (nb_flow * H * W)$ (hidden은 구현 선택)

예시(가정): `nb_flow=1, len_c=3, len_p=1, len_t=1, H=W=32, nb_filter=64, nb_res_units=12`

- 각 branch in_ch는 각각 3,1,1로 달라져 정확 값은 branch별로 약간 다르지만, res_unit이 대부분을 차지합니다(대략 0.9M/branch 수준). 논문 세팅이 “3×3 conv, 64 filters” 중심임을 근거로 합니다. ¹⁶

기존 IRVSTNet 파라미터 특징

IRVSTNet은 embedding + MultiheadAttention + BiLSTM + 소규모 linear로 이루어져 있으며, 특히 demand embedding의 vocab 크기(`max_demand`)에 따라 파라미터가 크게 변합니다. 이 값은 데이터 의존적 오픈 아이템입니다. ⁶ ²

FLOPs/메모리 근사(예시 + 공식)

ST-ResNet Conv FLOPs(대략)

Conv2D 한 층의 MACs는 대략 $B * H * W * out_ch * (in_ch * k * k)$ 입니다. 따라서 ST-ResNet은 `H, W`가 커질수록 **선형이 아니라 면적(HW) 비례로 급증**합니다. 이는 “현재 repo가 top-k cell만 사용해 계산량을 줄이는 전략”과 철학이 다릅니다. ²

기존 모델 메모리 특징

- IModule은 학습 파라미터는 없지만 `db_keys`가 `(N, Samples, T)`를 GPU에 올리므로, 데이터 길이가 길면 GPU 메모리 부담이 큼니다. ⁶
- ST-ResNet 기본안은 IModule을 제거하므로, DB 메모리는 사라지는 대신 conv feature map 메모리가 증가합니다.

요구사항 충족을 위한 예시 학습 호출

요구사항(사용자 요청 #9)의 “예시 training call”은 본 repo 구조상 `main.py` + hydra config가 표준이므로, 아래처럼 사용합니다. ²²

```
# ST-ResNet용 config를 추가했다고 가정 (configs/stresnet_base.yaml)
CUDA_VISIBLE_DEVICES=0 python main.py --config-name stresnet_base
```

Mermaid: 학습/추론 플로우 다이어그램

```
flowchart TD
    CFG[Hydra config] --> MAIN[main.py run()]
    MAIN --> DS[MyDataset (keyframe)]
    DS --> SPLIT[Subset split train/eval]
    SPLIT --> TR[Transformers Trainer]
    TR --> DL[DataLoader + collate_fn]
    DL --> BATCH["batch dict<br/>{demands_series, weather, time, labels, sample_idx}"]
```



```

BATCH --> WRAP[ModelTrainer.forward]
WRAP --> ST[STResNet.forward]
ST --> PRED[pred (B,N)]
WRAP --> LOSS[DMVSTLoss(pred, labels)]
LOSS --> BP[backprop/optimizer step]
TR --> CKPT[checkpoint save]
TR --> EVAL[compute_metrics/test_loop]

```

오픈 아이템 목록

마지막으로, 사용자 요구대로 “미정인 사항은 오픈 아이템”으로 명시합니다(확정 전에는 성능/자원 추산이 크게 틀릴 수 있음).

- `grid(7000).npy`의 실제 shape: `(T,H,W)` vs `(T,C,H,W)` vs 그 외
- 시간 간격: 1시간 단위인지(시간 feature 생성이 24를 가정하지만 확정 필요) ²
- 수요 채널 수: 단일 demand인지 inflow/outflow(2채널)인지(논문 기본은 2채널) ⁵
- period/trend 간격: $p=24, q=168$ 이 맞는지(데이터 단위에 따라 변경) ¹⁶
- weather/time 정렬: meteorological_data.csv의 row가 grid time index와 1:1 대응인지
- holiday/event feature 유무(없다면 external_dim은 weather+time으로 제한) ⁵
- $H \times W$ 가 큰 경우(예: 수천 $\times$ 수천) ST-ResNet이 현실적으로 가능한지(타일링/다운샘플링/희소화 필요 여부)

¹ ¹⁸ pipeline.md

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/docs/pipeline.md

² DemandDataset.py

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/models/loader/DemandDataset.py

³ ¹⁰ [1610.00081] Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction

<https://arxiv.org/abs/1610.00081>

⁴ main.py

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/main.py

⁵ ¹³ ¹⁴ ¹⁵ Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction

<https://arxiv.org/pdf/1610.00081.pdf>

⁶ GNVSTNet.py

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/models/GNVSTNet.py

⁷ transformers.trainer — transformers 4.11.1 documentation

https://huggingface.co/transformers/v4.11.1/_modules/transformers/trainer.html?utm_source=chatgpt.com

⁸ test.py

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/runners/test.py

⁹ cosine_base_ir.yaml

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/configs/cosine_base_ir.yaml

¹¹ ¹² ¹⁶ Predicting citywide crowd flows using deep spatio-temporal residual networks - ScienceDirect

https://www.sciencedirect.com/science/article/pii/S0004370218300973?utm_source=chatgpt.com

¹⁷ Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction - yjucho's blog

https://yjucho1.github.io/spatio-temporal%20data/deep%20learning%20paper/ST-resnet/?utm_source=chatgpt.com

19 20 **Trainer**

https://huggingface.co/docs/transformers/en/main_classes/trainer?utm_source=chatgpt.com

21 **Trainer**

https://huggingface.co/docs/transformers/v4.45.1/ko/trainer?utm_source=chatgpt.com

22 **readme.md**

https://github.com/greenClothesZelda/CustomDMVST/blob/Cosine_base_IR/readme.md